

HACKERRANK ALGORITHMS & GITHUB CODING PORTFOLIO

1. Student Information

Name: Prayag Kishore

USN: R25EF198

Program: B.Tech Computer Science & Engineering

Semester: 3rd Semester

Programming Language: C++

HackerRank Profile: <https://www.hackerrank.com/profile/prayagkishore05>

GitHub Repository: <https://github.com/prayagkishore05-droid/HackerRank-3rdSem-Algorithm-Portfolio>

2. Objective

The objective of this activity was to improve problem-solving and algorithmic programming skills by solving selected HackerRank problems and maintaining the solutions in a structured GitHub portfolio. The problems cover fundamental algorithmic techniques including linear traversal, insertion, binary search, sorting, and greedy approaches. The activity also focuses on understanding time and space complexity, comparing alternative approaches, documenting solutions, and maintaining evidence of successful submissions.

3. Problems Completed

Five algorithmic problems were solved using C++:

1. Mini-Max Sum — Linear Traversal — $O(N)$
2. Birthday Cake Candles — Linear Traversal — $O(N)$
3. Insertion Sort – Part 1 — Insertion/Element Shifting — $O(N^2)$ worst case
4. Binary Search / Intro to Tutorial Challenges — Binary Search — $O(\log N)$
5. Mark and Toys — Sorting + Greedy — $O(N \log N)$

4. Approach and Complexity Analysis

4.1 Mini-Max Sum

The array is traversed once while calculating the total sum, minimum value, and maximum value. The minimum sum is obtained by subtracting the maximum element from the total, while the maximum sum is obtained by subtracting the minimum element.

Time Complexity: $O(N)$. Space Complexity: $O(1)$ auxiliary space.

Alternative Approach: Sorting could identify the minimum and maximum elements, but it would require $O(N \log N)$ time. Linear traversal is more efficient.

4.2 Birthday Cake Candles

The maximum candle height and its frequency are calculated in a single traversal. Whenever a larger height is found, the maximum and count are updated. If the current height equals the maximum, the count is increased.

Time Complexity: $O(N)$. Space Complexity: $O(1)$ auxiliary space.

Alternative Approach: A frequency map could count all heights, but this requires additional memory.

4.3 Insertion Sort – Part 1

The last element is considered the value to be inserted. Elements larger than this value are shifted one position to the right until the correct position is found.

Time Complexity: $O(N^2)$ worst case. Space Complexity: $O(1)$.

Alternative Approach: Merge Sort or Quick Sort can provide better overall sorting performance, but this challenge specifically demonstrates the insertion process.

4.4 Binary Search

The sorted array is repeatedly divided into two halves. The middle element is compared with the target value. Depending on the comparison, either the left or right half is discarded.

Time Complexity: $O(\log N)$. Space Complexity: $O(1)$.

Alternative Approach: Linear search can find the target in $O(N)$ time, but binary search is more efficient for sorted arrays.

4.5 Mark and Toys

The prices are first sorted in ascending order. The cheapest toys are then purchased one by one until the available budget is insufficient.

Time Complexity: $O(N \log N)$. Space Complexity: $O(\log N)$ auxiliary space for the sorting operation, depending on the implementation.

Alternative Approach: A more complex selection-based approach could avoid full sorting in some cases, but sorting provides a simple greedy solution.

5. GitHub Portfolio and Evidence

All five C++ solutions have been organized in the GitHub repository using separate folders for each problem. The repository also contains an evidence folder containing screenshots of the HackerRank submissions.

Repository Structure:

```
HackerRank-3rdSem-Algorithm-Portfolio/  
├── README.md  
├── 01-Mini-Max-Sum/solution.cpp  
├── 02-Birthday-Cake-Candles/solution.cpp  
├── 03-Insertion-Sort-Part-1/solution.cpp  
├── 04-Binary-Search/solution.cpp  
├── 05-Mark-and-Toys/solution.cpp  
└── evidence/  
    ├── 01-Mini-Max-Sum-Accepted.png  
    ├── 02-Birthday-Cake-Candles-Accepted.png  
    ├── 03-Insertion-Sort-Part-1-Accepted.png  
    ├── 04-Binary-Search-Accepted.png  
    └── 05-Mark-and-Toys-Accepted.png
```

6. Reflection

This activity helped me improve my understanding of algorithms and their efficiency. While solving the five HackerRank problems, I learned that choosing the right algorithm can significantly reduce execution time and memory usage. The Mini-Max Sum and Birthday Cake Candles problems helped me practice linear traversal and updating values efficiently. Insertion Sort helped me understand how elements can be shifted to place a value in its correct position. Binary Search improved my understanding of dividing a search space into smaller sections and achieving $O(\log N)$ time complexity. Mark and Toys introduced me to the greedy approach, where choosing the cheapest available items allows the maximum number of toys to be purchased within a fixed budget.

Maintaining the solutions on GitHub also helped me understand how to organize programming work using folders, README documentation, and evidence screenshots. Overall, this activity improved both my algorithmic thinking and my ability to document and present programming work professionally.

7. Conclusion

The HackerRank Algorithms & GitHub Coding Portfolio activity was completed by solving five algorithmic problems in C++. The solutions demonstrate different techniques such as linear traversal, insertion, binary search, sorting, and greedy selection. Time and space complexities were analyzed for each solution, and alternative approaches were considered. The completed solutions, documentation, and HackerRank evidence have been organized in the GitHub repository.